Group 6: Methodology

Jun Tang
Fairleigh Dickinson University
Vancouver, Canada
j.tang4@student.fdu.edu

Xiaoyi Han
Fairleigh Dickinson University
Vancouver, Canada
x.han1@student.fdu.edu

Kaiweng Wang
Fairleigh Dickinson University
Vancouver, Canada
k.wang4@student.fdu.edu

Ziyan Qiu
Fairleigh Dickinson University
Vancouver, Canada
z.qiu1@student.fdu.edu

Hongyi Niu
Fairleigh Dickinson University
Vancouver, Canada
h.niu@student.fdu.edu

1 Methodology

1.1 Artifact and Environment Plan

We will follow the official Baleen-FAST 24 Guithub README to setup the environment to reproduce the work in the Baleen paper. The environment will include the pythong codes, data access. The artifact is archived in the github repository:

https://github.com/wonglkd/Baleen-FAST24.

The repository has the frozen code to reproduce simulated experiments done in the paper. and data download scripts, and jupyter notebooks for producing the results and plots.

The instructions are documented in the repository's README.MD, which provides details about: getting start guidance, experiment walkthrough, pointers to notebooks that plot paper figures, and directory structure.

Repository URL: https://github.com/wonglkd/Baleen-FAST24
Intended Commit ID: To be pinned later (currently unknown)
Target Notebooks: `chameleon/1-getting-started.ipynb`; `chameleon/2-start-dedicated-server.ipynb`; `notebooks/example/example.ipynb`; `notebooks/paper-figs/fig-01bc,17-202309.ipynb`; `notebooks/reproduce/commands.ipynb`
Directory Paths: `data/`, `runs/`, `tmp/`, `notebooks/`, `notebooks/figs/`

Table 1: Artifact and environment plan for Baleen-FAST24.

1.2 Repository Scope & Terminology

Scope. We will use the public parts of Baleen-FAST24: the Chameleon simulator and the set of Jupyter notebooks enumerated in the artifact table in §1.1. We will follow the default folders (`data/`, `runs/`, `tmp/`, `notebooks/`, `notebooks/figs/`). The exact code version is the one pinned in §1.1. Out of Scope: Anything not in the public repo is out of scope (internal testbed pieces, private traces, kernel/filesystem changes, cloud orchestration). Since A3 is a plan, we also omit install/build commands.

Assumptions. Our default setting is an HDD-backed flash cache with single-node simulation. We use offline training with a simple temporal split (train on earlier windows, evaluate on later windows), and enable coordinated ML prefetch when running Baleen.

Terminology (aligned with A1/A2). *DT*: backend disk service-time proxy we aim to reduce. *Peak DT*: tail/peak DT over an evaluation window. *Episode*: accesses to the same block grouped by LRU eviction age; used for OPT labels and imitation. *OPT*: oracle admission over episodes (uses future knowledge) to produce training labels for Baleen. *Segment granularity*: decision/accounting granularity (block/chunk/meta-feature level). *Service time*: per-miss backend latency linked to DT/Peak DT. *Flash write rate*: sustained write budget/constraint; a control knob in Baleen–TCO studies. *Cache hit rate* (diagnostic): secondary indicator to interpret DT changes; not the primary objective.

1.3 Planned Policies and Training

We plan to look at two caching policies. The first one is the baseline admission rule from the artifact notebooks. It is just a simple frequency or recency based rule and has no prefetch. We use this as our control so we know how much gain we get with ML admission. The second one is the Baleen ML policy. We will train admission by OPT imitation on episodes, same as in the Baleen paper [5]. This policy runs with coordinated prefetch and keeps a flash write-rate budget. The goal is to make DT smaller. Later we will try the model on a different part of the trace to see if it still works well.

Training will be offline. We split the trace by time: the early part for making episodes and labels, the later part for testing. This is so the model does not see future data by mistake. For now we just run the artifact scripts since they already handle episode creation and training. For features we start with everything: meta (request size, type), block (recency, frequency) and chunk (reuse distance, prefetch hints). If we still have time, we might drop some of them, like meta-only, and see what happens to DT.

Hyperparameters will follow the defaults first. After we get one model trained, we may try small changes, like making the learning rate half or double, just to check if training improves. We keep the model that makes DT smallest while staying under the write budget.

1.4 Planned Workloads and Temporal Splits

We will start with the Getting Started subset. This will help us check that the pipeline works. We will test episode creation, OPT label generation, model training, and DT extraction. After that, we will move on to other public subsets used in the artifact notebooks. We will pick at least one trace that has many small random reads, and one trace that has large or streaming reads. This will help us see different effects on DT and Peak DT.

The training will be offline. We will split the trace by time. We use the early part to create training data, and we use the later part

to test. This helps us avoid using future data by mistake. We will use the default settings from the notebooks. We will also follow the same split method as the paper. This helps us get results that match the Baleen setup. As a non-ML control, we use a recency/frequency baseline in the spirit of classic adaptive policies (e.g., ARC) before enabling Baleen's ML admission." [4].

1.5 Metric Definitions

In this subsection, we will define and plan the metrics.

Disk-head Time (DT) and Peak DT: DT represents the total time spent by an HDD head to handle missed requests in the flash cache, including the search, rotation and data transfer times. Peak DT represents the backend load in the worst-case scenario, the peaks of DT observed in a set of evaluation time Windows. Reducing DT, especially the peak DT, can directly optimize efficiency at the back end of the HDD, ensure stable I/O performance, and reduce delay. We will use the notebooks provided by artifact to obtain the DT and peak DT indicators, following the methods in the paper to calculate DT and peak DT from log data.

Flash write Rate: In subsequent experiments, we will adjust different target flash write rates to observe changes in back-end performance (such as peak DT) under different endurance by following the Balene-TCO methodology. We prioritize the flash write-rate first as [3] shows that effective admission can save flash wear without sacrifing performance.

Estimated TCO trendlines: The estimated TCO trend line will conceptually demonstrate the trade-off between cost and performance. We will link the write rates of different target flash memory with back-end performance (such as peak DT) and expected hardware costs, indicating that increasing the write rate may reduce peak DT, meanwhile, cost flash memory wear and overall costs.

Cache hit rate: Cache hit rate measures the proportion of requests processed directly from the flash cache instead of the HDD back-end. It is like a diagnostic indicator because a high hit rate does not necessarily mean a lower DT. Sometimes even a small number of misses may cause a significant backend load, thereby increasing DT. [1] also suggest not just aggregate hits.

1.6 Analysis and Reporting Plan

After we finish running the workloads, we will collect the DT and Peak DT numbers from each test window. We will report the median and IQR for these numbers. After that, we will repeat the test with different random seeds and add confidence intervals. We will compare Baseline and Baleen. Our focus is on peak DT. We will also report the cache hit rate. We will check that all runs stay under the given flash write rate. We will check that all runs stay under the given flash write rate. We will also write down how well each policy meets the write budget. This helps us see how faster speed may use more flash writes.

We also plan to test different flash write rates and cache sizes. This will help us understand how performance changes. We will draw charts like "Peak DT vs write rate" and "Peak DT vs cache size". If possible, we will also run tests without prefetch to compare results. If we have time, we will try simpler feature sets to see how that affects DT. This strategy also mirror [2].

Member Contributions

To start the paper, every team member read and understood the paper. Below are the contributions for each member:

Jun Tang (2111172)

- Read through the github artifacts.
- Wrote the Artifact and Environment Plan
- Draw the table of artifact and environment
- Tried out to clone the code to understand the structure of the code
- Set up latex tex files and coordinate team efforts.

Ziyan Qiu (2089937)

- Wrote §1.2 *Repository Scope & Terminology*.
- Listed what we use and what is out of scope.
- Set basic assumptions (HDD-backed flash cache, offline temporal split, ML prefetch).
- Aligned key terms with A1/A2.
- Fixed LaTeX refs/formatting (removed table ref, paths in); checked the build.

Kaiweng Wang (2073516)

- Studied the Baleen-FAST24 on GitHub and learned the structure.
- Wrote the section 1.2 Repository Scope & Terminology.
- Listed the default settings used in the experiments.
- Studied key terms like DT, Peak DT, and flash write rate.
- Matched terminology with earlier assignments (A1/A2).

Xiaoyi Han (2133542)

- Defined DT and Peak DT as primary backend performance metrics.
- Flash write rate as an endurance and Baleen-TCO control metric.
- Planned variation of target write rates to analyze performance and cost trade offs.
- Conceptually related write rate to Estimated TCO trendlines.
- Defined cache hit rate as a diagnostic metric but not the only criterion.

Hongyi Niu (2104640)

- Studied the Baleen-FAST24 on GitHub and learned the structure.
- Listed the workloads and directory paths used in the plan.
- Learned how the artifact creates episodes and OPT labels.
- Wrote the 1.4 Planned Workloads and Temporal Splits section.
- Wrote the 1.6 Analysis and Reporting Plan section.

References

[1] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. 2018. LHD: improving cache hit rate by maximizing hit density. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI '18)*. USENIX Association.

[2] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. 2020. The CacheLib caching engine: design and experiences at scale. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. USENIX Association.

[3] Assaf Eisenman, Asaf Cidon, Evgenya Pergament, Or Haimovich, Ryan Stutsman, Mohammad Alizadeh, and Sachin Katti. 2019. Flashield: a hybrid key-value cache that controls flash write amplification. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI '19)*. USENIX Association.

[4] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: a self-tuning, low overhead replacement cache. In *Proceedings of the 3rd USENIX Conference on File and Storage Technologies (FAST '03)*. USENIX Association.

[5] Lin-Kit Wong, Hao Wu, Chris Moler, Suriya Gunasekar, Jianshen Lu, Shaunak Khandkar, Ajay Sharma, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. 2024. Baleen: ML Admission and Prefetching for Flash Caches. In *Proceedings of the 22nd USENIX Conference on File and Storage Technologies (FAST 24)*. USENIX Association, Santa Clara, CA, USA. https://www.usenix.org/conference/fast24/presentation/wong